

DOCUMENTAÇÃO DO CÓDIGO

Organização do Projeto:

```
C:\project
├── healdb
│   ├── data
│   │   ├── input
│   │   │   ├── bulas
│   │   │   │   ├── categorias
│   │   │   │   │   ├── biologico
│   │   │   │   │   ├── dinamizado
│   │   │   │   │   ├── especifico
│   │   │   │   │   ├── fitoterapico
│   │   │   │   │   ├── generico
│   │   │   │   │   ├── novo
│   │   │   │   │   ├── prod_tp
│   │   │   │   │   ├── radiofarmaco
│   │   │   │   │   └── similar
│   │   │   ├── CID-10-CAPITULOS.csv
│   │   │   ├── CID-10-CATEGORIAS.csv
│   │   │   ├── CID-10-GRUPOS.csv
│   │   │   ├── CID-10-SUBCATEGORIAS.csv
│   │   │   ├── consulta_medicamento_anvisa.csv
│   │   │   ├── DADOS_ABERTOS_MEDICAMENTOS.csv
│   │   │   ├── input_active_ing_translate_meta
│   │   │   ├── lista_dcb.xlsx
│   │   │   ├── ddls
│   │   │   │   ├── healdb_hd_active_ingredient.sql
│   │   │   │   ├── healdb_hd_active_ingredient_ext_id.sql
│   │   │   │   ├── healdb_hd_company.sql
│   │   │   │   ├── healdb_hd_drug_interaction.sql
│   │   │   │   ├── healdb_hd_icd_category.sql
│   │   │   │   ├── healdb_hd_icd_group.sql
│   │   │   │   ├── healdb_hd_icd_subcategory.sql
│   │   │   │   ├── healdb_hd_medication.sql
│   │   │   │   ├── healdb_hd_medication_active_ingredient.sql
│   │   │   │   ├── healdb_hd_medication_disease.sql
│   │   │   │   ├── healdb_hd_medication_drug_leaflet.sql
│   │   │   │   ├── healdb_hd_regulatory_category.sql
│   │   │   │   ├── healdb_hd_symptom.sql
│   │   │   │   ├── healdb_hd_therapeutic_class.sql
│   │   │   │   └── healdb_hd_type_ext_id.sql
│   │   │   ├── ontologies
│   │   │   │   └── chebi.owl # RDF Schema ChEBI
│   │   ├── output
│   │   └── downloadBulas
```

```

| | | | translation
| | | | | output_active_ing_translate_meta.csv
| | | | interoperability
| | | | | chebi
| | | | | | chebi_attributes_healdb.csv
| | | | | | chebi_attributes_healdb_json.json
| | | | | iucn
| | | | | | iucn_conservation_dcb.json
| | | | | | iucn_conservation_healdb.json
| | | | | pubchem
| | | | | | pubchem_reference_healdb.json
| | | | | rxnorm
| | | | | | rxnorm_enrichment_healdb.json
| | | | rdf_schema
| | | | | healdb.ttl
| | | | | healdb_complete.ttl
|
| doc
| logs
| tests
| src
| | rdf_schema
| | | __init__.py
| | | main_rdf_schema.py
| | | create_healdb_rdf_schema.py      # Complete RDF schema for HealDB
| | | create_mini_healdb_rdf_schema.py # Mini RDF schema with instance data
|
| | interoperability
| | | __init__.py
| | | main_interoperability.py
| | | linking
| | | | __init__.py
| | | | import_dcb_data.py
| | | | populate_external_ids_types.py
| | | | external_ids_insert.py
| | | | link_cas_to_active_ing.py
| | | | link_rxcui_to_active_ing.py
| | | | link_rxcui_related_ids_to_active_ing.py
| | | | link_wikidata_ids_to_wrk_table.py
| | | | link_kegg_related_ids_to_active_ing.py
| | | | fill_missing_external_ids.py
| | | usecases
| | | | __init__.py
| | | | chebi_attributes_query.rd      # SPARQL query for CHEBI
| | | | iucn_export_conservation_status.py # IUCN integration
| | | | pubchem_export_references.py     # PubChem integration
| | | | rxnorm_export_enriched_ingredients.py # RxNorm integration

```

```

├── nlp_extraction
│   ├── __init__.py
│   ├── main_nlp_extraction.py
│   ├── extract_leaflet_sections.py
│   ├── translate_leaflet_indication.py
│   ├── extract_diseases_from_indications.py
│   ├── process_disease_data_json.py
│   └── link_medications_with_diseases.py
├── repositories
│   ├── __init__.py
│   ├── main_repository.py
│   ├── create_activeingr_repository.py
│   ├── create_diseases_repository.py
│   ├── create_drugbank_repository.py
│   ├── create_leaflets_repository.py
│   ├── create_medications_repository.py
│   ├── create_symptoms_repository.py
│   └── drugbank_inserts.py
├── translation
│   ├── __init__.py
│   ├── main_translation.py
│   ├── translate_active_ingredients.py
│   ├── translate_active_ingredients_meta.ipynb
│   ├── import_translated_active_ingredients_meta.py
│   ├── translate_drug_interactions.py
│   ├── translate_food_interactions.py
│   └── validate_translation_and_link_active_ing.py
├── webcrawler
│   ├── __init__.py
│   ├── main_webcrawler.py
│   └── webcrawler_leaflet.py
├── requirements.txt
├── config.py
├── db_utils.py
└── main.py

```

1. Pasta: c:\project\healdb\src\webcrawler

WEBCRAWLER_LEAFLET:

Objetivo: Automatizar a coleta e o armazenamento das bulas do Bulário Eletrônico da ANVISA, utilizando web crawling e web scraping para todas as categorias de bula.

Processo:

1. Automatização da navegação:
 - a) Utilizar a biblioteca **Selenium** para automatizar a navegação no Bulário Eletrônico da ANVISA, selecionando cada uma das 9 categorias de bula e acionando o botão de consulta correspondente.
2. Coleta de Dados:
 - a) Armazenar o conteúdo da tabela com os dados dos medicamentos em um DataFrame.
 - b) Percorrer todas as páginas dentro de cada categoria de bula.
 - c) Utilizar a biblioteca **BeautifulSoup** para realizar o web scraping da tabela contendo os dados das bulas em cada página.
3. Download e Armazenamento das Bulas:
 - a) Para cada URL de bula obtida (usando o driver do Selenium), realizar o download automático do arquivo da bula, que será salvo na pasta específica da categoria correspondente.
 - b) Atualizar o DataFrame com o nome do arquivo da bula e seu caminho (path) em PDF.
4. Registro e Arquivamento:
 - a) Após o processamento de todas as páginas de uma categoria, armazenar os dados do DataFrame em um arquivo Excel na respectiva pasta de categoria. Este arquivo Excel incluirá uma coluna com o caminho do arquivo PDF da bula.
5. Integração Posterior:
 - a) O arquivo Excel gerado será utilizado posteriormente pelo script `'create_leaflets_repository.py'` para integrar as bulas à base de dados.

Este processo assegura a coleta e organização sistemática das bulas, facilitando a integração e análise dos dados na base HealDB.

2. Pasta: c:\project\healdb\src\repositories

CREATE_MEDICATIONS_REPOSITORY:

Objetivo: Criar o repositórios de medicamentos.

Processo:

1. Utilizar fontes de dados externas: DADOS_ABERTOS_MEDICAMENTOS em formato csv do DataVisa.
2. Armazenar os dados nas seguinte tabelas do MySQL:
 - a) `'hd_medication'`: Armazenar informações sobre medicamentos.

- b) `'hd_therapeutic_class'` : Armazenar as classes terapêuticas.
 - c) `'hd_regulatory_category'` : Armazenar as categorias regulatórias dos medicamentos.
3. Processar os medicamentos:
- a) Extrair as classes terapêuticas e armazenar na respectiva tabela.
 - b) Extrair as categorias regulatórias e armazenar na respectiva tabela.
 - c) Armazenar os medicamentos e ligá-los com a classe terapêutica e categoria regulatória.

CREATE_DISEASES_REPOSITORY:

Objetivo: Criar o repositório de doenças.

Processo:

1. Utilizar fontes de dados externas: CID-10 em formato CSV.
2. Armazenar os dados nas seguintes tabelas do MySQL:
 - a) `'hd_icd_group'` : Armazenar dados sobre grupos de doenças do CID.
 - b) `'hd_icd_category'` : Armazenar dados sobre as categorias de doenças do CID.
 - c) `'hd_icd_subcategory'` : Armazenar dados sobre as subcategorias de doenças do CID.
3. Processar as doenças e inseri-las nas tabelas.

CREATE_SYMPTOMS_REPOSITORY:

Objetivo: Criar o repositório de sintomas usando dados hierárquicos do vocabulário controlado do BIREME/MeSH.

Processo:

1. Utilizar fontes de dados externas: BIREME/MeSH (vocabulário controlado de termos biomédicos que serve para indexar documentos e recuperar informações científicas) para os sintomas em formato csv.
2. Armazenar os dados na seguinte tabela do MySQL:
 - a) `'hd_symptom'` : Armazenar dados de sintomas.
3. Processar os sintomas e inseri-los de forma hierárquica na tabela.

CREATE_ACTIVEINGR_REPOSITORY:

Objetivo: Criar o repositório de princípios ativos.

Processo:

1. Ler os princípios ativos dos medicamentos armazenados na tabela `'hd_medication'`.
2. Armazenas os dados nas seguintes tabelas do MySQL:
 - a) `'hd_active_ingredient'` : Armazenar os dados de princípios ativos.
 - b) `'hd_medication_active_ingredient'` : Armazenar o relacionamento entre medicamentos e princípios ativos.
3. Processar os princípios ativos dos medicamentos:

- a) Dividir os princípios ativos no campo de princípio ativo da tabela usando o separador "+".
- b) Armazenar o resultado na tabela de trabalho `'hd_wrk_medication_active_ing'`, associando cada medicamento aos seus princípios ativos normalizados.
- c) A partir desta tabela de trabalho, armazenar os princípios ativos no repositório `'hd_active_ingredient'`.
- d) Estabelecer a correlação entre medicamentos e seus princípios ativos (usando o ID criado na tabela `active_ingredient`) e armazenar essa relação na tabela `medication_active_ingredient`.

CREATE_LEAFLETS_REPOSITORY:

Objetivo: Ler as pastas contendo as bulas baixadas do site da ANVISA em PDF, organizadas por categorias (biológico, dinamizado, específico, fitoterápico, genérico, novo, produto de terapia, radiofármaco e similar), e armazenar essas bulas na tabela `'hd_medication_drug_leaflet'` da base de dados MySQL.

Processo:

1. Para cada categoria de bula, ler o arquivo Excel com a lista de bulas.
2. Armazenar o conteúdo do arquivo Excel na tabela de trabalho `'hd_wrk_drug_leaflet'`.
3. Realizar a deduplicação das bulas, removendo as versões mais antigas e armazenando os dados na tabela de trabalho `'hd_wrk_drug_leaflet_dedup'`.
4. Carregar todos os registros da tabela `'hd_wrk_drug_leaflet_dedup'` em um DataFrame.
5. Para cada registro do DataFrame, ler o arquivo PDF da bula usando a biblioteca **PyMuPDF** (módulo **Fitz**). PyMuPDF é uma biblioteca Python de alto desempenho para extração, análise, conversão e manipulação de documentos PDF (e outros).
6. Para bulas no formato de imagem, utilizar a biblioteca **Pytesseract** para realizar a extração de texto. Python-tesseract é uma ferramenta de reconhecimento óptico de caracteres (OCR) para Python. Ou seja, ela reconhece e 'lê' o texto embutido em imagens.
7. Armazenar o conteúdo extraído no DataFrame e salvar na tabela `'hd_medication_drug_leaflet'`, incluindo o texto das bulas.
8. Realizar uma revisão adicional das bulas identificadas manualmente como imagens e tentar extrair o texto usando Pytesseract, se necessário.

CREATE_DRUGBANK_REPOSITORY:

Objetivo: Criar um repositório para os dados do DrugBank disponibilizados em um arquivo XML. Esses dados incluem informações sobre as drogas catalogadas no DrugBank, sinônimos, ingredientes do produto (formas salinas da droga), interações medicamentosas, interações entre drogas e alimentos. Os atributos coletados das drogas são: identificador no DrugBank, nome, descrição e indicação do uso da droga.

Processo:

1. Ler o arquivo XML do DrugBank.

2. Selecionar as tags: `'drugname-id'`, `'name'`, `'description'`, `'indication'`, `'food-interaction'`, `'drug-interaction'`, `'synonym'`, `'salt'`, `'external-id'`.
3. Armazenar em um DataFrame.
4. Criar o repositório do DrugBank chamando o script `'drugbank_inserts.py'` para inserir as informações que se encontram no DataFrame nas tabelas que compõem o repositório.

DRUGBANK_INSERTS:

Objetivo: Criar um repositório para os dados do DrugBank disponibilizados em um arquivo XML. Esses dados incluem informações sobre as drogas catalogadas no DrugBank, sinônimos, ingredientes do produto (formas salinas da droga), interações medicamentosas, interações entre drogas e alimentos. Os atributos coletados das drogas são: identificador no DrugBank, nome, descrição e indicação do uso da droga.

Processo:

1. Inserir as informações das drogas na tabela `'db_drug'`.
2. Inserir as interações das drogas com alimentos na tabela `'db_food_interaction'`.
3. Inserir os sinônimos das drogas na tabela `'db_synonym'`.
4. Inserir as informações das formas salinas das drogas na tabela `'db_product_ingredient'`.
5. Inserir as informações dos IDs externos que permitem interoperabilidade do drugbank com outras fontes de dados de saúde na tabela `'db_drug_external_id'`.
6. Inserir as informações das interações medicamentosas na tabela `'db_drug_interaction'`.
 - a. Pesquisar o ID da droga com a qual a droga principal interage, utilizando o `'id_DrugBank'`, que é um identificador imutável registrado no DrugBank.
 - b. Efetuar a inserção de todas as informações na tabela `'db_drug_interaction'` em modo batch, com o batch size igual a 2000 (inserção a cada 2000 registros).

3. Pasta: c:\project\healdb\src\translation

TRANSLATE_ACTIVE_INGREDIENTS:

Objetivo: Traduzir os princípios ativos para o inglês usando a API do OPENAI. Atualmente, utilizamos o modelo `'gpt-4-0314'`.

Obs: Os dois modelos `'gpt-4-0314'` e `'gpt-3.5-turbo-1106'` retornam o mesmo resultado. Caso seja necessário repetir o processo, recomenda-se o uso do `'gpt-3.5-turbo-1106'`, por ser mais leve e econômico.

Processo:

1. Copiar o conteúdo da tabela `'hd_active_ingredient'` para a tabela `'hd_translate_eng_active_ing'`, que armazenará as traduções dos princípios ativos.

2. Configurar as variáveis da API do OpenAI.
3. Ler os princípios ativos da tabela `'hd_translate_eng_active_ing'`.
4. Para cada princípio ativo, chamar a API do OpenAI utilizando o modelo `'gpt-4-0314'`.
5. Armazenar as traduções em um DataFrame.
6. Atualizar a tabela `'hd_translate_eng_active_ing'` com o princípio ativo traduzido para o inglês no campo `'nm_active_ingredient_eng_ini'`.
7. Além da tradução inicial, a tabela possui dois outros campos: um para revisão manual, se necessário (review), e outro para a tradução final (eng), que combina a tradução inicial com a revisão.

VALIDATE_TRANSLATION_AND_LINK_ACTIVE_ING:

Objetivo: Validar a tradução dos princípios ativos de duas formas:

- Verificar a correspondência do termo traduzido automaticamente (GPT-4.0) nas tabelas do DrugBank relacionadas a drogas, sinônimos e ingredientes dos produtos.
- Confirmar a correspondência do termo traduzido manualmente nas mesmas tabelas.

Processo:

1. Validar o termo traduzido automaticamente pela API da OpenAI (modelo `'gpt-4-0314'`) contra os dados do DrugBank.
2. Buscar o termo traduzido nas tabelas de drogas, sinônimos e ingredientes dos produtos, identificando os Ids correspondentes (`'id_drug'` e `'id_DrugBank'`).
3. Validar o termo traduzido manualmente contra os dados do DrugBank.
4. Armazenar a tradução final: usar a tradução manual se existir; caso contrário usar a tradução automática.
5. Inserir na tabela de princípios ativos X drogas (`'hd_priv_active_ingredient_drug'`) a relação entre o princípio ativo (em português) e a droga do DrugBank (em inglês).

OBS: essa relação entre o princípio ativo e a droga será utilizada para capturar as interações medicamentosas relacionadas com esse princípio ativo.

TRANSLATE_ACTIVE_INGREDIENTS_META:

Objetivo: Traduzir os princípios ativos para o inglês usando o modelo `'SeamlessMT4'` do META. Este script foi desenvolvido no google colab (notebook).

Processo:

1. Exportar o conteúdo da tabela `'hd_active_ingredient'` para um arquivo CSV.
2. Ler o arquivo CSV e armazenar em um DataFrame.
3. Efetuar a tradução usando o modelo SeamlessMT4.
4. Armazenar a tradução em um DataFrame.
5. Exportar o DataFrame com a tradução para um arquivo CSV.

IMPORT_TRANSLATED_ACTIVE_INGREDIENTS_META:

Objetivo: Importar a tradução do princípio ativo feita pelo modelo do META para comparar posteriormente com os dados do DrugBank.

Processo:

1. Importar o arquivo CSV contendo a tradução dos princípios ativos para a tabela `'hd_wrk_translate_eng_active_ing_meta'`.
2. Validar a tradução comparando o conteúdo da tabela `'hd_wrk_translate_eng_active_ing_meta'` com os medicamentos do DrugBank.
3. Computar o total de correspondências com o DrugBank.

OBS: O modelo GPT-4 conseguiu corresponder a 1.436 ingredientes ativos (60% do total) do repositório DrugBank, enquanto o modelo SeamlessM4T obteve apenas 143 correspondências (6% do total). Devido ao desempenho insatisfatório do modelo META, que apresentou erros significativos de tradução (por exemplo, traduzir "Insulina Glulisina" como "insuline and glutamine" ou "Óxido cúprico" como "Oxygenated copper"), decidiu-se utilizar o modelo GPT-4.0. As traduções foram validadas com base na correspondência correta encontrada no DrugBank. Por exemplo, embora as traduções "sulfate of sodium" e "sodium sulfate" estejam corretas, "sodium sulfate" foi considerada a forma correta com base no DrugBank.

TRANSLATE_FOOD_INTERACTIONS:

Objetivo: Traduzir as interações de alimentos do DrugBank do inglês para o português usando a API da OpenAI (modelo GPT-3.5) e enriquecer a base HealDB com essas informações.

Processo:

1. Inserir as interações de alimentos do DrugBank (`'db_food_interaction'`) na tabela de tradução (`'hd_priv_translate_port_food_int'`) cruzando com os princípios ativos existentes na base HealDB.
 - a. Ao inserir na tabela de tradução, substituir o nome da droga associada à interação por uma variável XXX (para possibilitar posteriormente a substituição dessa variável pelo princípio ativo em português já existente na base HealDB).
2. Enriquecer a base HealDB com as interações traduzidas.
 - a. Traduzir o campo que contém as interações com a variável XXX (`'ds_interaction_var'`) usando a API do OpenAI (modelo GPT-3.5). Traduzir as interações distintas, já que algumas se repetem devido ao uso da variável para o nome da droga, o que reduz o tempo de tradução. Foi escolhido o modelo GPT-3.5 porque o GPT-4.0 estava muito lento e a qualidade da tradução foi similar. Armazenar a informação traduzida em uma coluna da tabela, ainda com a variável XXX (`'ds_interaction_var_port'`). (Funções: `'perform_food_interaction_translation'` e `'update_translate_food_interaction'`).
 - b. Substituir a variável XXX pelo princípio ativo em português da base HealDB e salvar na tabela final que compõe a base de medicamentos, na tabela `'hd_priv_food_interaction'`. (Funções: `'replace_var_by_active_ingredient'` e `'insert_food_interaction'`)

Resultado: A base HealDB será enriquecida com as interações de alimentos:

- **A quantidade de interações resultantes foi 1.781.** Existem algumas interações repetidas, porque alguns princípios ativos são similares e se referem a mesma droga. Isso ocorre porque não foi feita a desambiguação dos princípios ativos, resultando em entradas duplicadas com descrições diferentes. No entanto, isso não compromete o resultado final de busca da interação a partir do MEDICAMENTO X PRINCÍPIO ATIVO. Se a desambiguação fosse realizada, a quantidade de interações seria 1.425, que é o total presente na tabela de tradução.

TRANSLATE_DRUG_INTERACTIONS:

Objetivo: Traduzir as interações medicamentosas da base original do DrugBank do inglês para o português usando a API do OpenAI (modelo GPT-3.5) e enriquecer a base HealDB com essas informações.

Processo:

1. Inserir as interações medicamentosas do DrugBank ('db_drug_interaction') na tabela de tradução ('hd_priv_translate_port_drug_int'), cruzando com os princípios ativos existentes na base HealDB. (Função: 'insert_table_translate_drug_int')
2. Ao inserir na tabela de tradução, substituir os nomes das drogas associadas à interação por variáveis XXX e YYY para possibilitar posteriormente a substituição dessas variáveis pelos princípios ativos em português já existentes na base HealDB.
3. Traduzir o campo que contém as interações com a variável XXX e YYY ('ds_interaction_var') usando a API do OpenAI (modelo GPT-3.5). Traduzir apenas as interações distintas, já que algumas se repetem devido ao uso das variáveis para os nomes das drogas, reduzindo assim o tempo necessário para tradução. O modelo GPT-3.5 foi o escolhido porque o GPT-4.0 estava muito lento e a qualidade da tradução foi similar. Armazenar a informação traduzida em uma coluna da tabela, ainda com as variáveis XXX e YYY ('ds_interaction_var_port'). (Funções: 'perform_drug_interaction_translation' e 'update_translate_drug_interaction')
4. Substituir a variável XXX e YYY pelos princípios ativos em português da base HealDB e salvar na tabela final que compõe a base de medicamentos, na tabela 'hd_drug_interaction'. (Funções: 'replace_var_by_active_ingredient' e 'insert_drug_interaction'). Devido à quantidade de registros das interações medicamentosas traduzidas, o insert foi realizado em lotes de 10.000 registros.

Resultado: A base HealDB será enriquecida com as interações medicamentosas:

- **A quantidade de interações resultantes foi 775.212.** Existem algumas interações repetidas, porque alguns princípios ativos são similares e se referem à mesma droga. Isso ocorre porque não foi feita a desambiguação dos princípios ativos, resultando em entradas duplicadas com descrições diferentes. No entanto, isso não compromete o resultado final da busca de interação a partir do MEDICAMENTO X PRINCÍPIO ATIVO. Se a desambiguação fosse realizada, a quantidade de interações seria 461.554, que é o total presente na tabela de tradução.

4. Pasta: c:\project\healdb\src\nlp_extraction

EXTRACT_LEAFLET_SECTIONS:

Objetivo: Percorrer o repositório de bulas, ler as bulas, extrair o texto que corresponde à seção "Para que este medicamento é indicado?" e "O que devo saber antes de usar esse medicamento?" de cada categoria de medicamento, e gravar o texto da indicação e da precaução no repositório de bulas.

Processo:

1. Ler a tabela de bulas `'hd_medication_drug_leaflet'` e armazenar em um DataFrame.
2. Iterar sobre cada registro do DataFrame que contém o conteúdo do repositório de bulas e armazenar o texto da indicação (vazio) no DataFrame `'df_indication'` e o texto da precaução(vazio) no DataFrame `'df_precaution'`.
3. Para cada bula do DataFrame, extrair a indicação utilizando uma expressão regular para encontrar o início e fim da sessão "Para que este medicamento é indicado" (biblioteca `re` do python).
4. Para cada bula do DataFrame, extrair a precaução utilizando uma expressão regular para encontrar o início e fim da sessão "o que devo saber antes de usar esse medicamento" (biblioteca `re` do python).
5. Inserir os dataframes `'df_indication'` e `'df_precaution'` em uma tabela de trabalho: `'hd_wrk_med_leaflet_section'`.
6. Atualizar a tabela `'hd_medication_drug_leaflet'` com o texto da indicação e da precaução.

TRANSLATE_LEAFLET_INDICATION:

Objetivo: Traduzir o texto da seção "para que este medicamento é indicado" das bulas para o inglês. O intuito desta tradução é utilizar ferramentas em inglês que conseguem extrair sintomas e doenças de textos médicos.

Processo:

1. Copiar as indicações da tabela `'hd_medication_drug_leaflet'` para a tabela `'hd_translate_eng_leaflet_section'`.
2. Utilizar para a API da OpenAI do modelo GPT-3.5 para a tradução.
 - Durante a tradução houve muitas interrupções, e continuei de onde parei. Mas a partir do dia 13/06/2024 às 17h56 até 14/06/2024 às 13h11, o processo seguiu sem interrupção.
3. Houve problemas na tradução dos seguintes `'id_medication'`:

- a) Atualizar como nula a tradução em inglês para esses IDs porque a origem ('ds_indication' da 'hd_medication_drug_leaflet') é nula: 671,1707, 1854, 1966, 2988, 3208, 3463, 3949, 4327, 4440, 4496, 5467, 5543, 5982, 5991, 6672, 7393, 9284, 9452, 9783.
- b) Repetir a tradução para esses IDs: 614, 1208, 2176, 4024.
 - i. Para o ID 614 houve a necessidade de atualizar o 'ds_indication' (em português) para remover um caracter especial que prejudicou a tradução.
 - ii. Para os demais IDs, a tradução foi submetida novamente e o resultado foi satisfatório.

EXTRACT_DISEASES_FROM_INDICATIONS:

Objetivo: Ler as indicações das bulas traduzidas para o inglês e utilizar o modelo pré-treinado Amazon Comprehend Medical para extrair as doenças e sintomas do texto das indicações.

Processo:

1. Para usar os serviços da Amazon Web Services (AWS), é necessário importar a biblioteca **Boto3** que é uma biblioteca de cliente para Python. O Boto3 fornece uma interface para interagir com os diversos serviços da AWS. Com essa biblioteca, é possível utilizar APIs, como a que realiza a extração de códigos CID a partir de textos médicos.
2. Ler a tabela 'hd_translate_eng_leaflet_section'.
 - a) Para cada registro da tabela, chamar a API 'comprehend_client_infer_icd10_acm' passando a coluna 'ds_indication_eng'.
 - b) Salvar a resposta da API em formato JSON na tabela 'hd_int_med_disease_api_response' para cada 'id_medication'.

PROCESS_DISEASE_DATA_JSON:

Objetivo: Extrair TODO o conteúdo do JSON (CID, descrição do CID e score) retornado pela API do Amazon Comprehend Medical (script 'extract_diseases_from_indications.py') e armazenar em uma tabela.

Processo:

1. Ler o conteúdo da tabela 'hd_int_med_disease_api_response'.
 - a) Transformar o string da resposta da API em um dicionário.
 - b) Extrair do dicionário diversas informações incluindo 'id_medication', 'cd_icd_full', 'ds_icd' (descrição do CID), 'vl_score' (probabilidade da informação estar correta).
 - c) Inserir essas informações na tabela 'hd_ind_med_disease_map'.

Tempo da extração: 16/06/24 das 12h14 às 12h38 (24 minutos).

LINK_MEDICATIONS_WITH_DISEASES:

Objetivo: Alimentar a tabela 'hd_medication_disease' com os CIDs extraídos (script 'process_disease_data_json.py'), estabelecendo uma correlação clara entre os

medicamentos e as doenças/sintomas. A nova tabela é populada com resultados de extração aprimorados, representando uma evolução significativa na pesquisa.

Processo:

1. Ler todos os medicamentos da tabela `'hd_int_med_disease_map'` cujos CIDs associados existam no repositório de CIDs do banco de dados HealDB (nas tabelas `'hd_icd_category'`, `'hd_icd_subcategory'`, `'hd_icd_group'`).
2. Criar um dicionário de medicamentos.
3. Para cada medicamento, filtrar os que possuem scores > 0,7.
 - a) Se os maiores scores tiverem 5 ou menos elementos, complementar com os 3 maiores scores desse medicamento (mesmo que estejam abaixo de 0,7).
 - b) Para os CIDs cujos scores foram selecionados:
 - i. Buscar nas tabelas `'hd_icd_category'` e `'hd_icd_subcategory'` a ligação com o `'cd_icd_full'` (sem o "ponto") efetuando join com as colunas `'cd_cat'` e `'cd_subcat'`.
 - ii. Inserir o medicamento, o `'cd_icd_full'` e a ligação com as tabelas `'hd_icd_group'`, `'hd_icd_category'` e `'hd_icd_subcategory'` na tabela `'hd_medication_disease'`.

OBS: 149 medicamentos não foram inseridos porque os CIDs gerados pelo Amazon Comprehend Medical não estão na tabela de CID-10 utilizada no Brasil e fonte das tabelas de doenças e sintomas do HealDB. Com o enriquecimento da tabela de CID do HealDB será possível inserir esses medicamentos que ficaram de fora.

Tempo do processamento: 16/06/24 das 16h34 às 16h37 (3 minutos).

5. Pasta: c:\project\healdb\src\interoperability\linking

Scripts para integração de dados de fontes externas por meio de external ids.

IMPORT_DCB_DATA:

Objetivo: Importar e processar os dados das Denominações Comuns Brasileiras (DCB) para o HealDB.

Processo:

1. Carregar os dados das DCB de um arquivo Excel.
2. Mapear e inserir classificações e descrições no banco de dados na tabela `'hd_dcb_classification'`.
3. Preencher a tabela `'hd_dcb_list'` com informações como números DCB, nomes, números CAS e histórico de classificação.

POPULATE_EXTERNAL_IDS_TYPES:

Objetivo: Popular a tabela `'hd_type_ext_id'` com os diferentes tipos de identificadores externos utilizados por diversas bases de dados na área da saúde.

Processo:

1. O script verifica se a tabela `hd\_type\_ext\_id` já contém registros.
2. Caso a tabela esteja vazia, ele inicia a etapa de inserção.
3. São carregados os seguintes identificadores externos, cada um com uma descrição curta e uma explicação mais detalhada:
 - CAS: CAS Number – Identificador numérico único atribuído pelo Chemical Abstracts Service.
 - RXCUI: RxNorm CUI – Identificador único de conceito do RxNorm para nomes normalizados de medicamentos.
 - PUBCHEM_CID: PubChem Compound ID – Identificador de compostos químicos no banco PubChem Compound.
 - KEGG_DRUG: KEGG Drug ID – Identificador de medicamentos no banco de dados KEGG Drug.
 - KEGG_COMP: KEGG Compound ID – Identificador de compostos químicos no KEGG Compound.
 - DRUGBANK: DrugBank ID – Identificador exclusivo de medicamentos na base DrugBank.
 - SNOMEDCT: SNOMED CT – Identificador de conceitos médicos nos termos clínicos do SNOMED.
 - CHEBI: ChEBI ID – Identificador de entidades químicas na base ChEBI.
 - ATC: ATC Code – Código da Classificação Anatômica Terapêutica Química.
 - UNII_CODE: UNII Code – Identificador único de ingrediente (Unique Ingredient Identifier) da FDA.
4. Cada identificador é inserido na tabela `hd\_type\_ext\_id` com os campos:
 - `tp\_ext\_id`: Código do tipo de identificador externo.
 - `ds\_short\_type`: Descrição curta.
 - `ds\_long\_type`: Descrição longa.

EXTERNAL_IDS_INSERT:

Objetivo: Inserir os IDs Externos nas tabelas do HealDB. É uma função chamada nos diversos scripts que efetuam a ligação entre external id's e o princípio ativo do HealDB, evitando repetição de código e centralizando em uma única função.

Descrição do processo:

1. Antes de inserir, valida se o tipo do “external id” existe na tabela `hd\_type\_ext\_id`.
2. Se não existir, interrompe o processo. Caso contrário, continua.
3. Insere o “external id” na tabela `hd\_active\_ingredient\_ext\_id`, apenas se ainda não existir esse código e tipo associado ao princípio ativo.

LINK_CAS_TO_ACTIVE_ING:

Objetivo: Associar os princípios ativos aos respectivos CAS Numbers da lista DCB, permitindo a interoperabilidade com bases e ontologias biomédicas externas.

Descrição do processo:

1. A tabela `hd\_active\_ingredient\_ext\_id` é limpa (TRUNCATE) para evitar duplicações.
2. A função `link\_cas\_from\_dcb\_to\_active\_ing\_ext\_id` vincula os CAS Numbers da tabela `hd\_dcb\_list` aos princípios ativos da `hd\_active\_ingredient`, com base na correspondência dos nomes.
3. São inseridos apenas CAS válidos (não nulos, sem [Ref...]), e ainda não vinculados.
4. Cada vínculo registra o tipo (CAS) e a origem (DCB).

LINK_RXCUI_TO_ACTIVE_ING:

Objetivo: Vincular os identificadores únicos de conceito do RxNorm (RXCUI) aos princípios ativos armazenados no HealDB, permitindo buscas e interoperabilidade com outras fontes e ontologias biomédicas.

Processo:

1. Ler os princípios ativos da tabela `hd\_translate\_eng\_active\_ing`, considerando apenas os que já possuem tradução final para o inglês.
2. Consultar a API do RxNorm utilizando o nome do princípio ativo em inglês para recuperar o identificador RXCUI correspondente.
3. Armazenar o identificador externo na tabela de interoperabilidade utilizando o tipo RXCUI e a origem RXNORM.
4. Registrar os dados na tabela `hd\_active\_ingredient\_ext\_id`, conectando o `id\_active\_ingredient` ao `cd\_ext\_id` (RXCUI).

LINK_RXCUI_RELATED_IDS_TO_ACTIVE_ING:

Objetivo: Vincular ao HealDB os identificadores externos relacionados ao RxNorm, como SNOMED CT, ATC, UNII e DrugBank, utilizando o identificador RxCUI previamente associado aos princípios ativos. Isso permite enriquecer a interoperabilidade com outras bases biomédicas e ontologias de saúde.

Processo:

1. Ler os princípios ativos da tabela `hd\_active\_ingredient\_ext\_id` que já possuem o identificador externo do tipo RXCUI.
2. Consultar a API do RxNorm para cada RxCUI, solicitando os seguintes identificadores relacionados: SNOMEDCT, ATC, UNII_CODE e DRUGBANK.
3. Para cada identificador externo encontrado, inserir na tabela `hd\_active\_ingredient\_ext\_id`, associando o `id\_active\_ingredient` ao valor retornado (`cd\_ext\_id`) com o respectivo tipo (`tp\_ext\_id`) e origem RXNORM.
4. Realizar chamadas com pequenos *delays* para respeitar os limites da API e evitar bloqueios.

LINK_WIKIDATA_IDS_TO_WRK_TABLE:

Objetivo: Buscar identificadores externos de fontes biomédicas no Wikidata utilizando os códigos RxCUI como entrada, armazenando os resultados em uma tabela de trabalho para posterior integração ao HealDB. Essa etapa visa enriquecer a base com links para ontologias como PubChem, ChEBI, KEGG, SNOMED CT e DrugBank.

Processo:

1. Buscar os princípios ativos do HealDB que já possuem o identificador externo do tipo RXCUI.
2. Agrupar os `rx cui` em lotes de 100 e consultar a API SPARQL do Wikidata para obter os identificadores externos relacionados.
3. Extrair os seguintes identificadores (quando disponíveis): CAS, DrugBank, PubChem, ChEBI, SNOMED CT, ATC, UNII, KEGG Compound e KEGG Drug.
4. Armazenar os resultados na tabela de estágio ``hd_wrk_wikidata_ext_id'`, garantindo que cada registro esteja vinculado ao `rx cui` correspondente e à URL do item no Wikidata (``ds_url_wikidata'`).
5. Verificar os tipos válidos de identificadores externos com base na tabela ``hd_type_ext_id'` antes de inserir.
6. Utilizar `executemany` para inserir os dados em lote, evitando duplicações e otimizando o desempenho.
7. Adicionar um pequeno *delay* entre os lotes para evitar bloqueio por excesso de requisições na API do Wikidata.

LINK_KEGG_RELATED_IDS_TO_ACTIVE_ING:

Objetivo: Conectar identificadores externos relacionados (PubChem CID e ChEBI ID) aos princípios ativos do HealDB, utilizando o identificador do KEGG (Compound) previamente obtido via Wikidata. Essa integração permite ampliar a interoperabilidade com outras fontes biomédicas abertas.

Processo:

1. Buscar os identificadores KEGG_COMP presentes na tabela ``hd_wrk_wikidata_ext_id'`, associados a princípios ativos do HealDB com RXCUI registrado.
2. Remover da tabela ``hd_active_ingredient_ext_id'` todos os registros com origem KEGG ou cujo tipo de identificador externo (``tp_ext_id'`) comece com KEGG.
3. Para cada identificador KEGG_COMP obtido, consultar a API pública do KEGG para recuperar os identificadores externos relacionados (PubChem CIDs e ChEBI ID).
4. Regravar o KEGG_COMP na tabela ``hd_active_ingredient_ext_id'` com origem WIKIDATA, e inserir os identificadores PubChem e ChEBI obtidos com origem KEGG.
5. Repetir o processo para todos os KEGG_COMP vinculados a princípios ativos, garantindo a vinculação completa entre os identificadores.

FILL_MISSING_EXTERNAL_IDS:

Objetivo: Preencher os identificadores externos ausentes na tabela

`hd\_active\_ingredient\_ext\_id`, utilizando os dados previamente carregados na tabela de trabalho `hd\_wrk\_wikidata\_ext\_id`, com base em correspondência pelo código RXCUI.

Processo:

1. Identificar os princípios ativos que possuem RXCUI e estão presentes em `hd\_wrk\_wikidata\_ext\_id`.
2. Verificar se já possuem o identificador externo correspondente cadastrado.
3. Inserir os novos identificadores externos que ainda não estão registrados do tipo CAS, ATC, SNOMEDCT, UNII_CODE, CHEBI and PUBCHEM CID.
4. Registrar a origem como WIKIDATA.

6. Pasta: c:\project\healdb\src\interoperability\usecases

Scripts para demonstrar os casos de uso de interoperabilidade do HealDB, explorando as conexões possíveis entre os princípios ativos do banco de dados e fontes externas de informação em saúde. Cada caso de uso mostra, de forma prática, como integrar o HealDB a bases abertas como ChEBI, PubChem, IUCN, entre outras, utilizando SPARQL ou APIs. Os resultados enriquecem os princípios ativos com atributos químicos, status de conservação, publicações científicas e outros dados biomédicos relevantes.

CHEBI_ATTRIBUTES_QUERY:

Objetivo: Demonstrar a interoperabilidade semântica entre o HealDB e a ontologia ChEBI por meio de uma query SPARQL executada no Apache Jena Fuseki. O caso de uso não corresponde a um script, mas sim a uma consulta SPARQL que cruza dois grafos RDF dentro do dataset

healdb_full:

- Grafo <<http://healdb.org/graph/extid>>: contém os princípios ativos do HealDB e seus identificadores externos.
- Grafo <<http://healdb.org/graph/chebi>>: contém a ontologia completa da ChEBI (arquivo chebi.owl).

Atributos retornados:

- ingredientLabel: nome do princípio ativo
- externalId: identificador externo (ChEBI ID)
- chebiEntity: URI do composto na ontologia ChEBI
- formula: fórmula molecular
- mass: massa molecular média
- monoisotopicMass: massa mono-isotópica
- smiles: notação estrutural SMILES
- inchi: identificador InChI
- inchikey: identificador InChIKey

PUBCHEM_EXPORT_REFERENCES:

Objetivo: Obter publicações científicas relacionadas a princípios ativos classificados como “antidepressivos” no HealDB, utilizando o identificador PubChem CID como chave de ligação com a base PubMed. A coleta é feita via APIs públicas da NCBI (Entrez E-Utilities), enriquecendo a base HealDB com referências científicas associadas a cada composto químico.

Processo:

1. Executar a consulta SQL na base HealDB para selecionar os princípios ativos associados à classe terapêutica “antidepressivo” que possuem identificador externo do tipo PUBCHEM_CID.
2. Para cada PubChem CID, consultar o endpoint `elink.fcgi` (Entrez API) para recuperar os PMIDs (identificadores de publicações no PubMed) relacionados.
3. Para cada PMID recuperado, obter os seguintes detalhes das publicações:
 - `title`: título da publicação
 - `journal`: periódico
 - `date`: data de publicação
4. Selecionar e ordenar as publicações por ano e mês (de forma aproximada) e manter as **três mais recentes**, com base na data retornada pela API (`PubDate`)..
5. Exportar o resultado em JSON, gravando o resultado no arquivo ``pubchem_reference_healdb.json'`, com os campos:
 - `id_active_ingredient`
 - `nm_active_ingredient`
 - `cd_pubchem_cid`
 - `publications`: lista de até três objetos contendo `pmid`, `title`, `journal`, `date`.

IUCN_EXPORT_CONSERVATION_STATUS_R:

Objetivo: Consultar o status de conservação, ameaças e distribuição geográfica de nomes relacionados a plantas medicinais (classificação DCB = "PM") por meio da API V4 da IUCN (International Union for Conservation of Nature), considerando tanto os nomes da lista DCB quanto os princípios ativos já registrados na base HealDB. Este script foi desenvolvido em R, diferentemente dos demais que foram desenvolvidos em Python.

Processo:

1. Recuperar os nomes da Denominação Comum Brasileira (DCB) classificados como "PM", independentemente de estarem ou não associados a um princípio ativo na base.
2. Recuperar os princípios ativos da base Healdb que estejam vinculados a DCBs classificados como "PM".
3. Normalizar os nomes científicos, removendo sufixos como nomes de autores e espaços invisíveis, mantendo apenas o gênero e espécie (ex: “*Aesculus hippocastanum*”).
4. Consultar a API da IUCN para obter:
 - Status de conservação (ex: "Vulnerável", "Em perigo")

- Ano de avaliação e escopo geográfico do assessment (Global, Europe, Mediterranean, Pan-Africa etc.)
- Informações sobre possível extinção
- Principais ameaças
- Localizações geográficas

A consulta retorna **todos os assessments disponíveis** para cada nome científico, incluindo os campos listados acima.

5. Armazenar os resultados em arquivos JSON, com resumos informando totais processados e com dados válidos.

Saídas:

- ``iucn_conservation_healdb.json``: Dados de conservação para os princípios ativos da base HealDB vinculados a DCBs do tipo "PM".
- ``iucn_conservation_dcb.json``: Dados de conservação para **todos** os nomes DCB classificados como "PM", mesmo que não estejam associados a um princípio ativo.

OBS: A API da IUCN foi acessada via R utilizando o pacote `iucnredlist`, que fornece uma interface de alto nível para os principais endpoints da Red List v4. Apesar do uso predominante de Python no projeto, a escolha pelo R se justifica pelo suporte oficial do pacote disponibilizado, facilidade de manipulação dos dados em estruturas `data.frame` e pelo fato da chamada via script Python não ter sido autorizada pela API. O script inclui um intervalo de 1,5 segundos entre chamadas para respeitar os limites da API.

RXNORM_EXPORT_ENRICHED_INGREDIENTS:

Objetivo:

Enriquecer os princípios ativos da base HealDB com informações clínicas e farmacológicas provenientes da API pública do RxNorm, utilizando o identificador RxCUI (RxNorm Concept Unique Identifier) previamente conectado à base. O enriquecimento inclui nome preferido, tipo de conceito, status, sinônimos e apresentações clínicas e comerciais, possibilitando análises mais completas e integração com outras ontologias biomédicas.

Processo:

1. Selecionar todos os princípios ativos que possuem identificador externo do tipo "RXCUI", conforme armazenado na tabela ``hd_active_ingredient_ext_id``.
2. Para cada princípio ativo com RxCUI, consultar a API pública do RxNorm para recuperar:
 - Nome preferido (preferred name)
 - Tipo de termo (TTY), como IN (Ingredient) ou PIN (Precise Ingredient)
 - Status do conceito e data de status
3. Recuperar os sinônimos associados ao RxCUI utilizando o endpoint de propriedades da API.

4. Consultar apresentações clínicas utilizando o TTY "SCD" (Semantic Clinical Drug), que representam combinações específicas de princípio ativo, dose e forma farmacêutica.
5. Consultar apresentações comerciais utilizando os TTYs "SBD", "BPCK", "SBDF" e "SBDC", por meio de chamadas separadas à API (uma para cada TTY).
6. Estruturar os dados enriquecidos em dicionários Python contendo todos os campos relevantes para cada princípio ativo.
7. Armazenar o resultado final em um arquivo JSON com todos os princípios ativos enriquecidos.

Saídas:

- ``rxnorm_enrichment_healdb.json``: Arquivo JSON contendo os dados enriquecidos para os princípios ativos com RxCUI, incluindo nome, status, sinônimos, apresentações clínicas e apresentações comerciais.

7. Pasta: c:\project\healdb\src\rdf_schema

CREATE_HEALDB_RDF_SCHEMA

Objetivo:

Gerar o RDF Schema completo do HealDB (sem instâncias) com base nas definições DDL do banco de dados relacional. O objetivo é representar estruturalmente todas as tabelas e suas relações (chaves estrangeiras) no formato RDF utilizando a biblioteca `rdflib`.

Processo:

1. Ler os arquivos DDL (`.sql`) a partir do diretório `PATHS["healdb_ddls"]`.
2. Extrair a estrutura do RDF Schema:
 - As colunas são identificadas como propriedades do tipo `owl:DatatypeProperty`, com inferência de tipo XSD (`string`, `integer`, `date`).
 - As chaves estrangeiras são representadas como `owl:ObjectProperty`, conectando classes distintas.
3. Criar as classes OWL:
 - Cada tabela é convertida em uma `owl:Class`, com rótulo (`rdfs:label`) correspondente ao nome.
4. Salvar o RDF Schema do HealDB no arquivo ``healdb_complete.ttl``, localizado em `PATHS["output_rdf_schema"]`.

Saída:

- ``healdb_complete.ttl``: Arquivo Turtle contendo o RDF Schema completo do banco de dados HealDB, sem dados instanciados.

CREATE_MINI_HEALDB_RDF_SCHEMA

Objetivo:

Gerar um RDF Schema instanciado do HealDB contendo apenas as entidades e relacionamentos relacionados aos princípios ativos (`ActiveIngredient`), identificadores externos

(ExternalIdentifier) e tipos de identificador (IdentifierType). O RDF inclui instâncias reais extraídas do banco de dados MySQL.

Processo:

1. Definir as Classes e Propriedades:
 - São criadas as classes `ActiveIngredient`, `ExternalIdentifier` e `IdentifierType`
 - São definidas as propriedades do tipo `owl:ObjectProperty` e `owl:DatatypeProperty` com seus domínios e ranges
2. Ler os dados do banco MySQL e instanciar o schema:
 - São lidas as tabelas ``hd_active_ingredient``, ``hd_active_ingredient_ext_id`` e ``hd_type_ext_id``.
 - Para cada entidade, são criadas instâncias com base nos dados extraídos.
 - Adiciona relações `owl:sameAs` para os identificadores externos do tipo CHEBI.
5. Salvar o RDF Schema simplificado do HealDB é salvo no arquivo `healdb.ttl`, localizado em `PATHS["output_rdf_schema"]`.

Saída:

- ``healdb.ttl``: Arquivo Turtle contendo o RDF Schema reduzido contendo um subconjunto do HealDB com dados instanciados, focado na interoperabilidade semântica com outras ontologias (ex.: ChEBI).